HIPAAChecker Testing Plan and Guideline

Table of Contents

Quick Links

1. HIPAAChecker Account: https://hipaachecker.health/sign-in
2. HIPAAChecker User Manual
3. HIPAAChekcer Testing Dataset Spreadsheet
4. Application Repository
5. HIPAAScoring CVSS Reports Spreadsheet
6. HIPAAChecker Testing Feedback Form

A. Executive Summary

This testing plan outlines the procedures, objectives, and methodologies for evaluating the HIPAAChecker platform, a tool designed to certify HIPAA technical safeguards in healthcare applications. The plan guides testers through assessing the accuracy and effectiveness of the HIPAAChecker in identifying HIPAA-related vulnerabilities across various application types. The testing process involves comparing HIPAA-compliant and non-compliant applications, validating scoring algorithms, and evaluating the usability and accuracy of the dashboard visualization and reporting features.

B. Background

HIPAAChecker Products Description:

HIPAAChecker certifies HIPAA technical safeguards of mHealth (Android and iOS) and web-based healthcare applications. The platform provides solutions for detecting and fixing HIPAA security and privacy-related vulnerabilities through the following components:

- DroidPortal & mPlugin—Android apps' vulnerability scanner and Android Studio plugin support
- xPlugin for iOS- xCode plugin to secure iOS Swift and Objective-C apps
- PyPortal- HIPAAChecker for Python Django web application
- JavaPortal- HIPAA Security Solution for Java Spring Boot web application
- PHPPortal- HIPAA Security Solution for PHP Laravel framework.
- RubyPortal- HIPAA Security Solution for Ruby on Rails
- JSPortal- HIPAA Security Solution for Express.js
- DotNETPortal - HIPAA Security Solution for .NET web application
- HIPAAChecker Mobile App- in-device mHealth app vulnerability checking for Android

HIPAAChecker scans source code and detects HIPAA-related vulnerabilities based on the following twelve rules:

Table 1: HIPAA Technical Safeguards

HIPAA Reference	Identifier	Technical Safeguards
164.312(a)(1)	authorization	Providing access controls to allow ePHI access only to persons or programs that have been granted access rights
164.312(a)(2)(i)	unique_id	Assigning unique id for identifying and tracking patient's identity
164.312(a)(2)(ii)	phi_emergency	Establishing procedures for obtaining necessary ePHI during emergency
164.312(a)(2)(iii)	user_inactivity	Implementing procedures to terminate a session after a predetermined time of inactivity
164.312(a)(2)(iv)	encryption_decryption	Implementing encryption and decryption of ePHI
164.312(b)	audit_control	Implementing audit controls to record and examine activity that contain or use ePHI
164.312(c)(1)	data_integrity	Maintain ePHI data integrity to prevent improper alteration or destruction
164.312(c)(2)	unauthorized	Mechanisms to corroborate that ePHI has not been altered or destroyed in an unauthorized manner
164.312(d)	user_authentication	Implement authentication procedures to verify that a person or entity seeking access to ePHI is the one claimed
164.312(e)(1)	guard_against_com_network	Implement technical security measures to guard against unauthorized access to ePHI that is being transmitted over communications network
164.312(e)(2)(i)	transmition_secuirity	Implement measures to ensure that transmitted ePHI is not improperly modified without detection until disposed of
164.312(e)(2)(ii)	phi_encryption	Encrypt ePHI whenever appropriate

Vulnerability Categories and HIPAAScoring Model:

HIPAAChecker scans source code to detect HIPAA-related vulnerabilities based on a comprehensive set of security rules aligned with HIPAA requirements. The platform's scoring system is based on the NIST standard Common Vulnerability Scoring System (CVSS) matrix and evaluates four primary vulnerability categories (VC):

VC1: Inadequate Data Security

- ➔ Data protection mechanisms
- ➔ Improper data encryption mechanisms
- ➔ Weak encryption of data at rest and/or in transit
- ➔ Weak cryptographic implementations
- ➔ Unauthorized data access and modification
- ➔ Unsecured data transmission across networks

Rule	Subrule	impact facto	IF value	count	related IF
	optimal_cryptographic_security	optimal	0.9	10	optimal
	strong_cryptographic_security	strong	0.75	1	negligible
	moderate_cryptographic_security	moderate	0.5	3	negligible
	minor_cryptographic_security	minor	0.25	8	negligible
	negligible_cryptographic_security	negligible	0.1	0	negligible
	optimal_hashing_security	optimal	0.9	5	optimal
	strong_hashing_security	strong	0.75	0	negligible
	moderate_hashing_security	moderate	0.5	2	negligible
	minor_hashing_security	minor	0.25	3	negligible
	negligible_hashing_security	negligible	0.1	58	negligible
	secure_data_storage	moderate	0.5	9	moderate
	code_bbfuscation_validation	moderate	0.5	37	moderate
encryption_decryption	custom_crypto_tink_encryption	moderate	0.5	0	negligible
	authorization_exception_for_dest	moderate	0.5	0	moderate
	illegal_destruction_restriction	moderate	0.5	120	moderate
	authorization_enforcement_for_d	moderate	0.5	286	moderate
	access_control_exception_for_de	moderate	0.5	1	moderate
authorization_for_destruction	biometric_authentication_for_des	moderate	0.5	11	moderate
unique_id	unique_id	strong	0.75	52	strong
	database_encryption	moderate	0.5	8	negligible
	api_base64_encode	minor	0.25	230	minor
phi_encryption	api_base64_decode	minor	0.25	186	minor
	authorization_exception_integrity	moderate	0.5	0	moderate
	illegal_access_integrity	moderate	0.5	120	moderate
	authorization_enforcement_integ	moderate	0.5	286	moderate
	access_control_exception_integri	moderate	0.5	1	moderate
	biometric_authentication_integrity	moderate	0.5	11	moderate
	input_validation	minor	0.25	0	minor
	user_authentication_oauth_integ	strong	0.75	25	negligible
	user_authentication_firebase_int	strong	0.75	1146	negligible
data_integrity	multi_factor_authentication_integ	optimal	0.9	2	optimal

VC2: Insufficient Authorization

- ➔ Missing essential headers in authorization forms, invalidate credentials
- ➔ Unable to verify authorization requests
- ➔ Weak authorization process evidence through inadequate access controls for PHI
- ➔ Insufficient mechanisms for validating and tracking user authorizations,
- ➔ Incomplete procedures for handling authorization renewals and revocations

Rule	Subrule	impact factor(IF)	IF value
unique_id	unique_id	strong	0.75
user_inactivity	user_inactivity	moderate	0.5
	user_authentication_oauth	strong	0.75
	user_authentication_firebase	strong	0.75
user_authentication	multi_factor_authentication	optimal	0.9
	authorization_exception	moderate	0.5
	illegal_access	moderate	0.5
	authorization_enforcement	moderate	0.5
	access_control_exception	moderate	0.5
authorization	biometric_authentication	moderate	0.5

VC3: Insecure Network Communication

- ➔ Unencrypted data transmission,
- ➔ Weak SSL/TLS implementations,
- ➔ Inadequate certificate validation mechanisms,
- ➔ Potentially exposing ePHI to man-in-the-middle attacks
- ➔ Network communications, including unverified hostname, acceptance of self-signed certificates, and unauthorized access to the ePHI during transmission

Rule	Subrule	impact factor(IF)	IF value
	secured_url_data_transmission	strong	0.75
	insecure_protocol_usage	negative_moderate	-0.5
	check_certificates_revocation	moderate	0.5
transmition_secuirity	authorized_api_call	strong	0.75
	trusted_server_communication	strong	0.75
	insecure_server_communication	negative_moderate	-0.5
	server_certificate_pinning	strong	0.75
	disabled_certificate_validation	negative_strong	-0.75
	custom_ssl_validation	strong	0.75
	disabled_hostname_verification	negative_moderate	-0.5
	strict_hostname_validation	minor	0.25
	secured_webview_communicatio	moderate	0.25
guard_against_com_network	secured_webview_content	minor	0.25

VC4: Inconsistent Audit Trail

- ➔ Missing critical events logs or maintaining incomplete logs of ePHI access and modifications
- ➔ User authentication events, data access patterns, and information sharing activities

Rule	Subrule	impact factor(IF)	IF value
	data_access_audit	none	0
	data_modification_audit	none	0
	security_event_audit	none	0
	audit_integrity_temper_protectior	none	0
	user_event_tracking_audit	optimal	0.9
	application_life_cycle_audit	negligible	0.1
	application_performance_audit	minor	0.25
audit	policy_enforcement_violation_det	none	0

HIPAA Risk Score Calculation:

The HIPAA Risk Score is calculated based on the frequency of detected vulnerable code segments. The scoring system follows the CVSS approach, dividing the security risk score into:

- **Risk Mitigation Score**: Based on implemented security measures
- **Risk Remaining Score**: Representing residual risk after mitigation

The total CVSS score for each vulnerability category (VC1-VC4) was predetermined using the CVSS calculator. The risk mitigation score has been calculated based on the implementation of specific security requirements (HIPAA Rule-> HIPAA Subrule) in the codebase. Here, Total CVSS Score = Risk Mitigation Score + Risk Remaining Score.

Check an example HIPAA Risk Score calculation.

Data description:

Column A: **Vulnerability** contains VC1-VC4

Column B: **Rule** contains HIPAA rule references i.e. encryption_decryption refere HIPAA Technical Safeguard 164.312(a)(2)(iv)

Column C: **Subrule** break down of each HIPAA Rule into multiple vulnerability patterns

Column D: **Impact Factor(IF)** refer the importance of a pattern for the specific HIPAA Rule

Column E: **IF value** contains a numerical weight of the importance

Column F: **Count** contains the frequency of a pattern found in the codebase

Column G: **Relative IF (RIF)** refers how this pattern affected by other vulnerabilities

Column H: **RIF Value** a numerical weight of the relative importance

Column I: **Factor** calculated based on the frequency of a pattern found

Column J: **Cumalative Factor** summation of IF and RIF

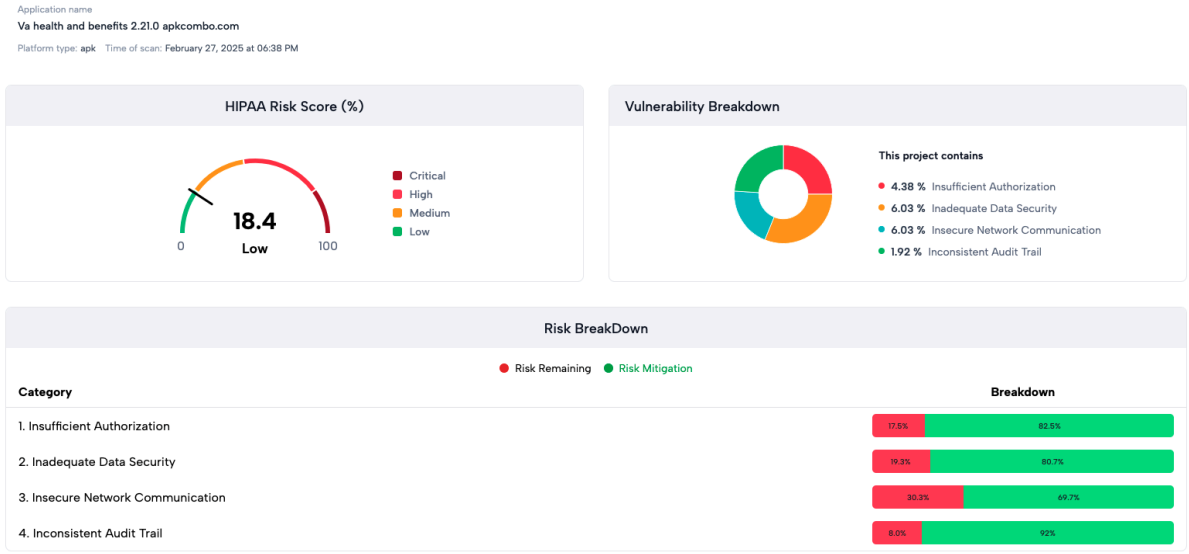
Column K: **Mitgated** contains the Risk Mitigation Score

Column L: **Remaining** contains Risk Remaining Score

The complete summarized HIPAAScore is placed at the bottom of each sheet

Vulnerability	CVSS Score	Mitigated	Risk remaining	HIPAA Risk Score
Insufficient Authorization	7.3	6.54742268	0.7525773196	10.31%
Inadequate Data security	9.1	7.862719298	1.237280702	13.60%
Insecure Network communica	5.8	4.960526316	0.8394736842	14.47%
Inconsistent Audit Trail	7	6.44	0.56	8.00%
Total	**29.2**	**25.81066829**	**3.389331706**	**11.61%**

A sample HIPAAScore visualization on the HIPAAChecker dashboard:



C. Testing Objectives

1. Dashboard Visualization Testing:

- Verify accuracy and usability of HIPAA Risk Score gauge representation, Risk level categorization (Low, Medium, High, Critical), and Percentage calculations and display
- Validate correct calculation of vulnerability percentages, categorization of vulnerabilities, and distinction between different vulnerability types

2. HIPAAScoring Accuracy Validation:

- Perform accuracy validation of the HIPAAScoring algorithm by comparing scoring matrix between HIPAA-Compliant and HIPAA-Non-Compliant applications
- Validate scoring accuracy against the benchmark remarks
- Verify if the tool correctly identifies and prioritizes security vulnerabilities

3. HIPAA Report Testing:

- Test the HIPAA report description and mapping with the HIPAA guidelines
- Verify completeness and accuracy of vulnerability details in reports
- Test description clarity of vulnerabilities reported in the dashboard
- Test filtering and sorting functionalities in the reporting interface
- Validate handling of incomplete or corrupted scan data for each supported application type

D. Testing Environment Setup

Prerequisites

- Computer with internet access
- Modern web browser (Chrome, Firefox, or Edge recommended)
- Basic understanding of application security concepts
- Familiarity with healthcare applications or HIPAA requirements (preferred)

Account Setup

1. Create a testing account at https://hipaachecker.health/sign-in
2. Verify email address and complete profile setup if required
3. If needed, contact nishat@ubitrix.com to increase the App upload permissions

E. Testing Methodology

Testing Approach

The testing process follows a comparative analysis approach, evaluating both HIPAA-compliant and non-compliant applications to validate the tool's ability to differentiate between them. Testing will be conducted in three phases:

1. **HIPAA-Compliant Application Testing**: Testing applications known to implement proper HIPAA safeguards
2. **HIPAA-Non-Compliant Application Testing**: Testing applications with known HIPAA vulnerabilities
3. **User-Selected Application Testing:** Testing applications selected by the tester to validate real-world applicability

Testing Steps

1. Tool Familiarization

7. Create an account at https://hipaachecker.health/sign-in
8. Review the HIPAAChecker User Manual to learn how to upload applications and generate reports
9. Explore the dashboard interface to understand available features and functions

2. HIPAA-Compliant Application Testing

1. Select a HIPAA-Compliant application from the HIPAAChekcer Testing Dataset Spreadsheet
    - You may choose any application from the Android, Django, Laravel, DotNet, Spring, Ruby, or ExpressJS sheets

- The **Label** column indicates HIPAA-Compliant or HIPAA-Non-Compliant apps
- The **Benchmark Remarks** column indicates the security features of the application

2. Download the chosen application from the Application Repository (search by **App ID**)
3. Upload the Application:
    - Navigate to the HIPAAChecker Dashboard
    - Select "New Scan"
    - Upload the application file or provide GitHub URL if applicable
    - Initiate the scan process
    - Allow the system to complete the analysis (timing may vary by application size)
4. Analyze Results:
    - Review the generated HIPAA Risk Score
    - Examine the vulnerability breakdown by category
    - Explore the detailed findings in the HIPAA report
    - Review the implementation of security features mentioned in the Benchmark Remarks
    - Review the vulnerabilities detection for unknown issues not mentioned in the Benchmark Remarks
    - Compare the scoring matrix from the HIPAAScoring CVSS Reports Spreadsheet (search by **App ID**) with the Benchmark Remarks of the application

3. HIPAA-Non-Compliant Application Testing

1. Select a HIPAA-Non-Compliant application from the HIPAAChekcer Testing Dataset Spreadsheet
2. Follow the same download and upload process as in Step 2-3 of HIPAA-Compliant Application Testing
3. Analyze Results:
    - Review the generated HIPAA Risk Score
    - Examine the vulnerability breakdown by category
    - Explore the detailed findings in the HIPAA report
    - Verify that vulnerability detection aligns with known issues mentioned in the **Benchmark Remarks**
    - Compare the scoring matrix from the HIPAAScoring CVSS Reports Spreadsheet (search by **App ID**) with the **Benchmark Remarks** of the application

4. User-Selected Application Testing

1. Select an application from your organization or one that you are familiar with
    - Ideally, choose an application with healthcare-related functionality
2. Upload the Application:
    - Follow the same upload process as in previous testing steps

- If using a GitHub repository, ensure you have appropriate access permissions

3. Analyze Results:
   - Review the complete HIPAA report
   - Assess whether identified vulnerabilities match your knowledge of the application

F. Reporting and Survey Completion

Compete the following Google Form survey based on your testing results:

[HIPAAChecker Testing Feedback](https://docs.google.com/forms/d/e/1FAIpQLSd--vDgbUZA19KkPKfa6Mfc8QuhAhbojt0r5lkTxPEHsmml8Q/viewform?usp=header) (https://docs.google.com/forms/d/e/1FAIpQLSd--vDgbUZA19KkPKfa6Mfc8QuhAhbojt0r5lkTxPEHsmml8Q/viewform?usp=header)

Contact Information

For technical support or questions during testing:

Email: nishat@ubitrix.com

Glossary

- **CVSS**: Common Vulnerability Scoring System, a framework for rating the severity of security vulnerabilities
- **ePHI**: Electronic Protected Health Information
- **HIPAA**: Health Insurance Portability and Accountability Act
- **NIST**: National Institute of Standards and Technology
- **PHI**: Protected Health Information
- **SSL/TLS**: Secure Sockets Layer/Transport Layer Security, protocols for secure network communications
- **VC1-VC4**: Vulnerability Categories 1-4 as defined in the HIPAAChecker system